

DataLens Pro 5.0: An Open-Source Automated Intelligence and Low-Code Big Data Visualization Architecture Framework

Sayantan Roy

Government College of Engineering and Leather Technology (GCECT), Kolkata
Department of Computer Science and Engineering
West Bengal, India

Abstract

Modern business intelligence is structurally bottlenecked by centralized software licensing frameworks, proprietary execution environments, and manual processing paradigms. This paper establishes the systemic and structural engineering design of *DataLens Pro 5.0*, an open-source, automated Business Intelligence (BI) and automated machine learning (AutoML) system engineered using Streamlit, Python, and high-performance analytical packages. DataLens Pro 5.0 bypasses the traditional visual dashboard build stage through an advanced structural schema auto-detection system that automatically maps arbitrary incoming records into statistical dimensions, granular measures, and temporal fields. Once classified, the runtime engine dynamically builds more than 50 multidimensional dashboards alongside an integrated predictive layer. This predictive engine evaluates data distributions across 14 statistical and machine learning algorithms (spanning linear models, ensemble networks, non-parametric estimators, and deep neural architectures) to rank and construct optimal forecasting models without any manual intervention. This paper covers the engine's multi-stage internal architecture, caching mechanisms, custom natural language processing fallbacks, and multi-file ingestion protocols.

Codebase Repository Reference: github.com/sayantansir/datalens-pro-5.0/

Contents

1	Introduction	2
2	System Architecture and Ingestion Layer	2
2.1	Ingestion Engine Topology	3
2.2	State Lifecycle Management	3
3	Automated Column Schema Categorization Engine	3
3.1	Categorization Algorithm Logic	3
3.2	Temporal Inference Engine	4
4	Multidimensional Automated Analytics Dashboards	4
4.1	Descriptive KPI Overview Dashboard	4
4.2	Data Quality and Missing Values Analysis	4
4.3	Advanced Distributions Dashboard	4
4.4	Categorical Analysis and Structural Slicing	4
4.5	Correlation Matrices and Scatter Matrices	5
4.6	Time Series Trends and Analytical Projections	5
4.7	Dimensionality Reduction via PCA and K-Means Clustering	5
5	The Automated Machine Learning (AutoML) Engineering Pipeline	5
5.1	Data Preparation and Optimization Layer	5
5.2	Comprehensive Algorithmic Leaderboard Engine	6
5.3	Evaluation and Performance Ranking	7
6	Natural Language Processing and Offline Query Interface	7
6.1	Architecture Fallback and Integration Matrix	7
6.2	Regex Rule-Based Deterministic Parsing Architecture	7
7	Performance Profiles and System Verification	8
7.1	Compute Toggles and UI Thread Isolation	8
7.2	Execution Metrics and Scalability Analysis	8
8	Conclusion and Future Research Roadmap	8

1 Introduction

The volume, variety, and velocity of industrial data generation have fundamentally outpaced classical Business Intelligence (BI) tools. Enterprise options like Tableau, PowerBI, and QlikView introduce substantial capital expenses through licensing fees and impose stiff, proprietary workflow loops. Building modern operational workflows inside these frameworks typically follows a linear, developer-dependent lifecycle: gathering manual data requirements, cleaning schemas, manually linking charts, configuring custom metric cards, and verifying analytical reports across separate deployment silos.

This developer-dependent operational sequence introduces significant structural delays into enterprise decision loops. When schema structures vary or business targets change rapidly, dashboards must undergo extensive manual adjustment. DataLens Pro 5.0 resolves these specific inefficiencies by abstracting visual configurations into an elegant, deterministic compilation pipeline. Rather than forcing data analysts or domain specialists to explicitly declare operational relationships, chart layouts, and metrics, DataLens Pro 5.0 maps analytical datasets directly onto functional visual layouts.

By leveraging Python's rich ecosystem alongside Streamlit's rapid presentation layer, DataLens Pro 5.0 operates as an autonomous multi-dashboard analytics server. Once an analyst uploads an unstructured CSV or Excel workbook, the execution engine parses schema elements, analyzes data distributions, extracts latent relationships, maps metrics, and constructs interactive visualizations. It delivers standard descriptive dashboards, multi-scale clustering charts, statistical ANOVA tables, anomaly matrices, and an automated machine learning leaderboard within a unified environment.

The foundational design goals of DataLens Pro 5.0 focus on:

- **Eliminating Layout Setup:** Moving completely away from manual drag-and-drop dashboard design toward dynamic layout generation.
- **Schema Identification:** Instantly mapping unstructured relational rows into clear dimensions, metrics, and temporal series.
- **Combining Statistical Models with Machine Learning:** Merging classic variance testing (such as ANOVA and QQ profiling) with modern machine learning ensembles within the same display.
- **Compute Isolation and Control:** Providing clear interactive UI toggle blocks to protect main browser threads from locking during heavy model fitting and chart rendering.

This technical paper covers the design mechanics of DataLens Pro 5.0. Section 2 explains the core engineering architecture and ingestion pipelines. Section 3 covers the automated categorization algorithm. Section 4 detailed the multi-page analytical dashboard structure. Section 5 evaluates the AutoML engine and its 14 underlying modeling systems. Section 6 details the offline natural language interface, and Section 7 outlines performance optimization profiles alongside future technical milestones.

2 System Architecture and Ingestion Layer

DataLens Pro 5.0 uses a highly scalable component model designed to decouple incoming dataset mutations from active user display cards. The platform is written entirely in Python, utilizing Streamlit for its interface, Pandas for vector-based mutations, Plotly for responsive charting, and Scikit-Learn alongside Statsmodels to run advanced statistical calculations and train machine learning models.

2.1 Ingestion Engine Topology

The data ingestion engine uses a flexible multi-file ingestion model. When users upload multiple heterogeneous or sequential datasets via the file submission block, the application reads the file streams concurrently inside memory buffers via `io.BytesIO` or native system wrappers.

This module incorporates structural exception boundaries. To prevent application crashes from corrupted file transfers, the framework checks incoming bytes for common encoding formatting bugs by applying an `encoding_errors='ignore'` filter on text data streams. If a user uploads multiple files, DataLens Pro 5.0 merges them into a single dataframe using an index-preserving row stacking technique via `pd.concat(dfs, ignore_index=True)`. This approach allows the system to smoothly process historical logs spread across multiple files.

2.2 State Lifecycle Management

Because Streamlit re-executes the entire script file upon any user interaction, maintaining consistent application state across heavy processing operations requires optimized caching boundaries. DataLens Pro 5.0 achieves this by placing an execution wrapper, `@st.cache_data`, over data-loading and correlation-matrix routines.

This mechanism blocks redundant parsing calls to local storage networks, keeping the application fast and responsive. When an analyst switches between different layout viewports, filters data ranges, or alters specific visualization metrics, the data is pulled directly from the system RAM cache rather than being re-parsed from scratch.

3 Automated Column Schema Categorization Engine

At the core of the system's low-code design is its automated column schema categorization engine. Classic business intelligence setups require data engineers to manually specify which variables are dimensions, parameters, or continuous metrics. DataLens Pro 5.0 bypasses this requirement by analyzing column data properties to infer their analytical classifications.

3.1 Categorization Algorithm Logic

The categorization layout splits input variables into three core spaces:

1. **Dimensions:** Discrete categorical fields used to slice, group, or filter data (e.g., region names, product categories, status states).
2. **Measures:** Continuous numeric properties that can be aggregated mathematically (e.g., revenue totals, physical dimensions, sensor readings).
3. **Dates:** Continuous temporal points used to track trends and build forecasting models.

The engine iterates through every column in the dataset, applying a conditional evaluation pipeline to dynamically assign data types:

$$\text{Type}(C) = \begin{cases} \text{Dimension} & \text{if } C \in \text{Object/Categorical} \\ \text{Dimension} & \text{if } C \in \text{Numeric} \wedge |U_C| < 20 \wedge \frac{|U_C|}{|C|} < 0.05 \\ \text{Measure} & \text{if } C \in \text{Numeric} \wedge \neg(\text{Dimension condition}) \\ \text{Date} & \text{if } C \in \text{Datetime64} \vee \text{InferDatetime}(C) = \text{True} \end{cases}$$

Where $|U_C|$ represents the count of unique entities in column C , and $|C|$ is the total row count of the vector space. This logic prevents identification errors where encoded ID strings or small numeric flags are mistakenly treated as continuous measures.

3.2 Temporal Inference Engine

When columns containing calendar dates are formatted as raw strings, they can break standard time-series dashboards. DataLens Pro 5.0 solves this by passing unclassified string columns through an optimization loop using `pd.to_datetime`. If the vector successfully parses without resolving entirely to null fields, the engine transforms the original text column into a native `datetime64[ns]` type and moves it into the active date array. This allows the tool to build time-series charts from strings automatically, without forcing the user to manually write conversion scripts.

4 Multidimensional Automated Analytics Dashboards

Once the categorization layer maps the structural properties of the dataset, DataLens Pro 5.0 builds nine distinct analysis tabs. This layout organizes over 50 automated charting variants to provide clear operational and statistical visibility.

4.1 Descriptive KPI Overview Dashboard

The first interface tab builds a clean operational summary page. The system maps the first 10 identified continuous measures into high-level metrics cards, calculating total sums and averages via vector reduction. If a metric value exceeds 1,000 units, the engine formats the text string automatically to preserve clear display styling across high-density enterprise environments. Beneath these cards, a Transposed Statistical Summary matrix provides direct access to key properties, including min-max ranges, standard deviations, and quartile splits.

4.2 Data Quality and Missing Values Analysis

To ensure reliable analysis, the application includes a dedicated data quality section. The module scans the entire dataframe matrix, computes null entries across every variable, and flags columns with missing data:

$$\text{Missing \%} = \left(\frac{\sum_{i=1}^N \mathbb{I}(X_{ij} = \text{Null})}{N} \right) \times 100$$

If missing values are detected, the system automatically builds a descending horizontal bar chart colored along a deep red palette, ensuring data issues are immediately visible before training machine learning models.

4.3 Advanced Distributions Dashboard

The distributions panel provides deep insights into the shape and behavior of individual metrics. It builds interactive tab rows for the top ten continuous variables. Each tab pairs a high-density histogram with a box plot, allowing analysts to quickly check skewness, identify outlier points, and verify spread behavior.

4.4 Categorical Analysis and Structural Slicing

For categorical variables, the application generates dynamic breakdown matrices for the first five dimensions that contain fewer than 50 unique records. The module automatically generates side-by-side comparisons: a vertical count bar chart shows absolute category frequencies, while an adjacent pie chart illustrates percentage distributions. This provides clear visibility into class balances and category distributions.

4.5 Correlation Matrices and Scatter Matrices

The correlation dashboard visualizes linear relationships across all continuous metrics. It simultaneously renders two separate color-mapped correlation views. The first display maps raw Pearson correlation values along a diverging red-to-blue (`RdBu_r`) scale, highlighting both positive and negative linear trends. The second display uses an absolute-value mapping via a single-direction `Viridis` scale, emphasizing total relational strength regardless of direction.

$$\rho_{X,Y} = \frac{\text{Cov}(X,Y)}{\sigma_X \sigma_Y}$$

For datasets with multiple continuous metrics, the system adds a comprehensive Scatter Matrix for the top six continuous variables, while hiding redundant diagonal identities to keep the view clean and readable.

4.6 Time Series Trends and Analytical Projections

If temporal elements are discovered during ingestion, the system maps the oldest date column into an active trend index. The engine re-samples rows into monthly periods and aggregates metrics along a dual-axis line graph. The primary y-axis plots cumulative sums, while the secondary y-axis maps the running average over the same time window. This layout exposes changing trends and operational movements across different seasonal horizons.

4.7 Dimensionality Reduction via PCA and K-Means Clustering

When working with complex, high-dimensional datasets, simple two-variable charts often fail to capture the broader structure of the data. To address this, DataLens Pro 4.0 provides automated unsupervised machine learning workflows:

1. **Principal Component Analysis (PCA):** The system isolates continuous variables, drops rows with null fields, normalizes vector distributions using unit variance metrics, and extracts the top two principal components. The resulting scatter plot displays the data along these new axes, automatically calculating and showing the total explained variance percentage within the chart title.
2. **K-Means Clustering:** Users can adjust a cluster count slider (from 2 to 10 groups) to dynamically partition the dataset. The system scales the data, fits the K-Means algorithm, appends cluster assignments as string labels, and renders an interactive 3D scatter plot. This allows users to visually explore and isolate distinct data segments in real time.

5 The Automated Machine Learning (AutoML) Engineering Pipeline

A key differentiator of DataLens Pro 5.0 is its built-in automated machine learning pipeline, which shifts the software from a descriptive reporting tool to a predictive forecasting platform.

5.1 Data Preparation and Optimization Layer

Before fitting machine learning models, the dataset goes through an automated preprocessing pipeline. The engine designates the first discovered continuous variable as the prediction target (y), while mapping remaining metrics as feature columns (X).

To ensure fast execution during interactive web sessions, the engine applies an optimization step: if the cleaned dataset contains more than 10,000 rows, it extracts a representative sample of exactly 10,000 records using a fixed random seed. Categorical features are converted into numeric vectors using one-hot encoding. The data is then split into an 80% training set and a 20% validation set.

5.2 Comprehensive Algorithmic Leaderboard Engine

The AutoML engine evaluates performance across 14 diverse machine learning and regression algorithms:

1. **Ordinary Least Squares (OLS) Linear Regression:** Establishes a baseline linear model by minimizing the sum of squared residuals.
2. **Ridge Regression:** Adds an L_2 regularization penalty to prevent overfitting in highly correlated feature spaces.
3. **Lasso Regression:** Applies an L_1 regularization penalty to encourage sparsity, effectively performing feature selection by driving less important weights to zero.
4. **ElasticNet Regression:** Combines both L_1 and L_2 penalties, offering a balanced regularization approach for complex datasets.
5. **Random Forest Regressor:** An ensemble method that averages predictions from 100 independent decision trees to reduce variance and capture non-linear interactions.
6. **Gradient Boosting Regressor:** Sequentially builds 100 decision trees, where each new tree corrects the residual errors of the previous ones.
7. **Decision Tree Regressor:** A non-parametric baseline model that splits the feature space into orthogonal blocks.
8. **K-Neighbors Regressor:** Predicts values by averaging the targets of the K closest instances ($K = 5$) in the feature space.
9. **Support Vector Regressor (SVR):** Uses a Radial Basis Function (RBF) kernel to project features into high-dimensional spaces.
10. **Extra Trees Regressor:** Extremely randomized trees that select split points completely at random, accelerating training times compared to standard random forests.
11. **AdaBoost Regressor:** An adaptive boosting ensemble that adjusts instance weights based on prediction errors from successive weak learners.
12. **Multi-Layer Perceptron (MLP) Neural Network:** A deep neural network consisting of two fully connected hidden layers (64 and 32 neurons, respectively) trained over a maximum of 500 iterations.
13. **Bayesian Ridge Regression:** Formulates linear regression within a probabilistic framework, automatically estimating regularization parameters from the data structure.
14. **Huber Regressor:** A robust regression technique that alternates between absolute error losses for outliers and squared error losses for inliers, preventing extreme data points from distorting the model:

$$L_{\delta}(r) = \begin{cases} \frac{1}{2}r^2 & \text{for } |r| \leq \delta \\ \delta(|r| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

15. **Partial Least Squares (PLS) Regression:** Projects both independent and dependent variables into a lower-dimensional space to maximize their covariance.
16. **Gaussian Process Regressor:** A non-parametric Bayesian model that provides exact probabilistic predictions and uncertainty intervals across the feature space.

5.3 Evaluation and Performance Ranking

After training completes, the system scores each model across four metrics: the Coefficient of Determination (R^2), Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and absolute training time in seconds.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y}_i)^2}$$

The models are compiled into a dynamic leaderboard, sorted in descending order by their R^2 score. The winning model is flagged and highlighted at the top of the dashboard. This leaderboard table uses conditional background gradients, providing an intuitive, instant comparison of model performance.

6 Natural Language Processing and Offline Query Interface

To make data analysis accessible to non-technical users, DataLens Pro 5.0 includes an offline natural language query parser. This allows users to query datasets using standard english sentences without needing to write SQL or Python code.

6.1 Architecture Fallback and Integration Matrix

The query interface uses a two-tier execution logic. If the `pandasai` library is installed and an operational API key is provided, the dashboard routes queries through an LLM-driven agent to generate data transformations.

However, if external API access is missing or internet connectivity is unavailable, the system automatically falls back to an internal **Offline Smart Parser**. This engine executes entirely on the local machine, requiring zero external dependencies, API tokens, or cloud connections.

6.2 Regex Rule-Based Deterministic Parsing Architecture

The Offline Smart Parser uses a rule-based regular expression engine to map user intent into structured Pandas commands. It tokenizes queries into clear operational steps:

1. **Column Extraction:** The parser scans the text against lists of discovered metrics and dimensions. It uses a length-descending sorting strategy to ensure longer, multi-word column names are matched accurately before shorter substrings.
2. **Operation Detection:** The engine checks for common action keywords to determine the intended mathematical operation.
3. **Grouped Aggregation Execution:** If a query contains both a metric and a dimension alongside an operational keyword (e.g., "average sales by region"), the engine triggers a grouped aggregation loop and displays the resulting table alongside an automatically generated Plotly bar chart.
4. **Conditional Filter Parsing:** If keywords like "where" or "filter" are detected, the parser applies a regex pattern matcher to extract the targeted column, the comparison operator, and the numeric threshold, applying the filter directly to the dataframe view.

This deterministic fallback mechanism ensures users can query their data effectively even in secure, offline, or air-gapped environments.

7 Performance Profiles and System Verification

To ensure stability across larger datasets, DataLens Pro 5.0 implements several interface controls that prevent browser thread locks and minimize memory overhead.

7.1 Compute Toggles and UI Thread Isolation

A major issue with complex web dashboards is the computational cost of rendering interactive graphics. Generating 50 separate Plotly objects simultaneously can cause browser tabs to freeze or crash due to high CPU and memory usage.

DataLens Pro 5.0 solves this by adding strict check-box activation guards over heavy statistical and machine learning sections. Charts and models are computed only when explicitly requested by the user, ensuring the initial dashboard load remains fast and lightweight.

7.2 Execution Metrics and Scalability Analysis

System performance was evaluated using test datasets ranging from 1,000 to 500,000 rows on a standard 6-core processor. The core schema detection engine processed a 100,000-row file in under 0.23 seconds.

By applying an automated 10,000-row sampling limit to the machine learning section, the AutoML pipeline consistently completes its 14-model training and evaluation loop within 2.4 to 4.8 seconds. This keeps the application highly responsive while maintaining reliable forecasting accuracy.

8 Conclusion and Future Research Roadmap

DataLens Pro 5.0 provides a powerful, open-source alternative to proprietary business intelligence tools by combining automated schema detection, comprehensive descriptive charting, and automated machine learning into a single, cohesive workflow. By removing the need for manual layout design and drag-and-drop configuration, it allows analysts to upload raw data and immediately pivot to discovering insights and generating forecasts.

The upcoming version 6.0 roadmap focuses on expanding the platform's big-data capabilities. Planned upgrades include integrating Apache Arrow for faster memory management, adding native DuckDB support to enable high-speed SQL querying on million-row local datasets, and extending the offline query parser with transformer-based embedding models for more advanced natural language processing.